OMNIX

Decision Governance Infrastructure

Schema Specification — Velos Integration Document

Schema v6.5.4e | ADR-022, ADR-031, ADR-043, ADR-044 | T=0 Gateway Edition

For	Naimat Khan — Velos
Prepared by	Harold Nunes — OMNIX QUANTUM LTD
Date	April 09, 2026
Classification	Confidential — Partner Integration Material
Schema Version	v6.5.4e
Engine Version	6.5.4e
Signing Standard	NIST FIPS 204 — ML-DSA (Dilithium-3 / Dilithium-5)
KEM Standard	NIST FIPS 203 — ML-KEM (Kyber-768)
Purpose	T=0 Interceptor Gateway — Velos POST endpoint integration

Every decision emitted by OMNIX produces a Post-Quantum Decision Receipt — a cryptographically signed JSON object encoding the complete governance trail of a single enforcement decision. This document defines the exact schema Velos must implement to correctly verify receipts at its T=0 interceptor gateway.

1. Overview

The OMNIX receipt is the atomic unit of the evidence layer. Each receipt is:

- Signed using NIST FIPS 204 — Dilithium-3 (ML-DSA-65) at enterprise baseline, or Dilithium-5 (ML-DSA-87) at high-assurance tier
- Chained — each receipt includes the SHA-256 hash of its predecessor, forming a tamper-evident audit trail
- Time-bounded — carries explicit ttl_epoch_ms for T=0 gateway enforcement
- Domain-agnostic — identical schema for trading, Islamic credit, insurance, and robotics

ENDPOINT: POST https://velos-gateway.onrender.com/api/v1/intercept All OMNIX receipts sent to this endpoint must pass TTL check + hash integrity + PQC signature verification before any downstream action is triggered.

2. Complete JSON Field Reference

```
{
"receipt_id": "OMNIX-A3F7C1D92B0E",
"timestamp": "2026-04-09T14:23:01.847291+00:00",
"issued_at_ms": 1744208581847,
"ttl_epoch_ms": 1744208611847,
"ttl_ms": 30000,

"asset": "BTC/USDT",
"decision": "BLOCK",

"veto_chain": [
"AML: PASS — volume_score=12.4 frequency_score=0.3",
"FRAUD: PASS — sentiment_score=0.12 reversal_risk=0.04",
"SHARIA: BLOCK — gharar_score=0.81 exceeds threshold 0.35",
"CAG: SKIPPED (upstream veto)"
],

"policy_version": "6.5.4e",
"engine_version": "6.5.4e",
"prev_hash": "a9f3c1d4e8b2fc71...",
"signing_provider": "dilithium3",
"content_hash": "3f8e7a2c91bd...",

"signature": "BASE64_PQC_SIGNATURE==",
"signature_algorithm": "Dilithium-3 (ML-DSA-65)",
"signature_format": "base64_pqc",
"public_key": "BASE64_PUBLIC_KEY==",

// Conditional — only present if gate was evaluated:
"sharia_compliance": { ... },
"aml_compliance": { ... },
```

```
"fraud_compliance": { ... },
"jurisdiction_compliance": { ... },
"context_admission": { ... },
"veto_type": "SHARIA_BLOCK",
"avm_result": { ... }
}
```


3. Identity Fields (always present)

Field	Type	Description
`receipt_id`	*string*	Globally unique receipt identifier. Format: OMNIX-{12 hex chars uppercase}. Example: OMNIX-A3F7C1D92B0E
`timestamp`	*string*	ISO-8601 UTC timestamp of receipt generation. Example: 2026-04-09T14:23:01.847291+00:00

4. Timing Fields — T=0 Gateway Critical (always present from v6.5.4e)

These three fields are the primary mechanism for Velos T=0 enforcement. They are part of the signed payload (included in content_hash). A receipt is expired when: now_ms > ttl_epoch_ms

Field	Type	Description
`issued_at_ms`	*integer*	Unix epoch milliseconds when the receipt was generated. Derived from timestamp. This is the canonical T=0 reference point for the decision.
`ttl_epoch_ms`	*integer*	Unix epoch milliseconds when the receipt expires. ttl_epoch_ms = issued_at_ms + ttl_ms. Velos MUST reject any receipt where now_ms > ttl_epoch_ms.
`ttl_ms`	*integer*	Time-to-live in milliseconds. Default: 30000 (30 seconds). Configurable on OMNIX side via OMNIX_RECEIPT_TTL_MS environment variable.

Velos T=0 ingestion rule:

```
now_ms = int(time.time() * 1000)
if now_ms > receipt['ttl_epoch_ms']:
return {'status': 'REJECTED', 'reason': 'RECEIPT_EXPIRED'}
```

5. Decision Fields (always present)

Field	Type	Description
`asset`	*string*	Asset symbol. Examples: BTC/USDT, ETH/USD, UNKNOWN
`decision`	*string*	Final governance decision, uppercased. Values: BLOCK | ALLOW | PASS | UNKNOWN. BLOCK means OMNIX has vetoed the action — Velos must not proceed.
`veto_chain`	*array[string]*	Ordered list of gate verdicts. Each entry format: 'GATE_NAME: RESULT — detail'. Maximum 80 characters per entry. Empty array if no trace available.

6. Versioning Fields (always present)

Field	Type	Description
`policy_version`	*string*	Policy version that evaluated this decision. Usually matches engine_version.
`engine_version`	*string*	OMNIX engine version. Current production version: 6.5.4e
`prev_hash`	*string*	SHA-256 content_hash of the immediately preceding receipt in the chain. Empty string for the first receipt. Forms the tamper-evident chain.

7. Cryptographic Fields (always present)

Field	Type	Description
`signing_provider`	*string*	Identifier of the active signing algorithm. Values: dilithium3 | dilithium5 | ed25519 | sha256. Bound into the signed payload — prevents algorithm confusion and downgrade attacks. Velos MUST dispatch verification based on this field.
`content_hash`	*string*	SHA-256 hex digest of the canonical JSON of all mandatory + present conditional fields, serialized with sort_keys=True, ensure_ascii=True. This is what gets signed.
`signature`	*string|null*	Detached digital signature over content_hash.encode('utf-8'). Encoding depends on signature_format — see Section 8.
`signature_algorithm`	*string*	Human-readable algorithm name. Examples: Dilithium-3 (ML-DSA-65), SHA-256, NONE.
`signature_format`	*string*	Critical for Velos ingestion. Disambiguates the encoding of the signature field. See Section 8 for all possible values and how to handle each.
`public_key`	*string|null*	Base64-encoded public key for asymmetric signature verification. null in SHA-256 fallback mode (signature_format = hex_sha256_fallback).

8. signature_format — Dispatch Table

This field is the key switch for how Velos must process the signature. Never assume — always read signature_format first.

Value	Meaning	Verification
`base64_pqc`	signature is Base64-encoded raw PQC signature bytes.	base64.b64decode(signature) → verify with Dilithium/ML-DSA using public_key
`hex_sha256_fallback`	signature is hex of SHA-256(content_hash). Symmetric. No asymmetric signing. public_key will be null.	sha256(content_hash.encode()).hexdigest() == signature
`NONE`	Signing failed. Receipt has no integrity guarantee.	**REJECT this receipt unconditionally.**

9. Conditional Governance Blocks

These blocks appear only when the corresponding gate was evaluated for the decision. Structure varies by domain and gate version.

Field	Present when
`sharia_compliance`	Islamic finance gate was evaluated
`aml_compliance`	AML gate was evaluated
`fraud_compliance`	Fraud gate was evaluated
`jurisdiction_compliance`	Jurisdiction gate was evaluated
`context_admission`	CAG (Context Admission Gate) was evaluated

`veto_type`	context_admission is present and contains a veto_type string
`avm_result`	AVM (Assumption Validity Monitor) was evaluated

avm_result sub-schema:

```
{
"is_valid": true,
"snapshot_id": "snap-abc123",
"parameter_version": "v3.1",
"drift_score": 0.12,
"age_hours": 2.4,
"pass_through": false,
"block_reason": "DRIFT_EXCEEDED" // only if AVM blocked
}
```

10. content_hash Contract — What Gets Hashed

The content_hash is computed before signing:

```
canonical = json.dumps(payload_fields, sort_keys=True, ensure_ascii=True)
content_hash = sha256(canonical.encode('utf-8')).hexdigest()
```

Mandatory fields in hash:	Conditional (if present):
asset	aml_compliance
decision	avm_result
engine_version	context_admission
issued_at_ms (new v6.5.4e)	fraud_compliance
policy_version	jurisdiction_compliance
prev_hash	sharia_compliance
receipt_id	veto_type
signing_provider	**NOT included in hash:**
timestamp	content_hash (itself)
ttl_epoch_ms (new v6.5.4e)	signature
ttl_ms (new v6.5.4e)	signature_algorithm
veto_chain	signature_format
	public_key

11. Full Verification Algorithm for Velos

Drop this function directly into the Velos interceptor. No external dependencies beyond pypqc (pip install pypqc).

```
import json, hashlib, base64, time

def verify_omnix_receipt(receipt: dict) -> dict:
result = {'receipt_id': receipt.get('receipt_id'), 'valid': False, 'reason': None}

# Step 1 — TTL check
ttl_epoch_ms = receipt.get('ttl_epoch_ms')
if ttl_epoch_ms is not None:
now_ms = int(time.time() * 1000)
if now_ms > ttl_epoch_ms:
result['reason'] = 'RECEIPT_EXPIRED'
return result

# Step 2 — Reconstruct payload for hashing
MANDATORY = ['receipt_id', 'timestamp', 'issued_at_ms', 'ttl_epoch_ms', 'ttl_ms',
'asset', 'decision', 'veto_chain',
'policy_version', 'engine_version', 'prev_hash', 'signing_provider']
CONDITIONAL = ['sharia_compliance', 'aml_compliance', 'fraud_compliance',
'jurisdiction_compliance', 'context_admission', 'veto_type', 'avm_result']
```

```
payload = {k: receipt[k] for k in MANDATORY if k in receipt}
if payload.get('signing_provider') is None:
payload.pop('signing_provider', None)
for k in CONDITIONAL:
if k in receipt:
payload[k] = receipt[k]

# Step 3 — Verify content_hash
canonical = json.dumps(payload, sort_keys=True, ensure_ascii=True)
computed_hash = hashlib.sha256(canonical.encode('utf-8')).hexdigest()
if computed_hash != receipt.get('content_hash'):
result['reason'] = 'HASH_MISMATCH'
return result

# Step 4 — Verify signature based on signature_format
sig_format = receipt.get('signature_format', 'UNKNOWN')
signature = receipt.get('signature')
public_key = receipt.get('public_key')
message = receipt['content_hash'].encode('utf-8')

if sig_format == 'base64_pqc':
provider = receipt.get('signing_provider', 'dilithium3')
if provider == 'dilithium3':
from pqc.sign import dilithium3
try:
dilithium3.verify(base64.b64decode(signature), message,
base64.b64decode(public_key))
result['valid'] = True
except Exception:
result['reason'] = 'SIGNATURE_INVALID'
elif provider == 'dilithium5':
from pqc.sign import dilithium5
try:
dilithium5.verify(base64.b64decode(signature), message,
base64.b64decode(public_key))
result['valid'] = True
except Exception:
result['reason'] = 'SIGNATURE_INVALID'

elif sig_format == 'hex_sha256_fallback':
expected = hashlib.sha256(receipt['content_hash'].encode()).hexdigest()
result['valid'] = (expected == signature)
if not result['valid']:
result['reason'] = 'FALLBACK_HASH_MISMATCH'
```

```
else:

result['reason'] = f'UNKNOWN_SIGNATURE_FORMAT:{sig_format}'

return result
```

12. Cryptographic Assurance Tiers

Tier	Provider ID	Algorithm	Standard	Security	PK Size	Sig Size
Enterprise Baseline (default)	`dilithium3`	Dilithium-3 (ML-DSA-65)	FIPS 204	~192-bit	1952 B	~3309 B
High-Assurance	`dilithium5`	Dilithium-5 (ML-DSA-87)	FIPS 204	~256-bit	2592 B	~4627 B
Dev/Test Fallback	`ed25519`	Ed25519 (classical)	—	128-bit	32 B	64 B
No PQC Available	`sha256`	SHA-256 (symmetric fallback)	—	N/A	N/A	64 hex

Tier is set on OMNIX side via PQC_SIGNING_LEVEL env var (3 or 5). Velos MUST dispatch verification based on the signing_provider field embedded in the receipt — do not hardcode a provider.

13. Receipt Chaining — Tamper-Evident Audit Trail

Each receipt carries prev_hash = content_hash of the immediately preceding receipt:

```
Receipt[n].prev_hash == Receipt[n-1].content_hash
```

A chain break indicates:

- A receipt was injected out of order
- A receipt was tampered with
- Chain was restarted — first receipt always has prev_hash = ""

14. Integration Checklist — Velos T=0 Gateway

#	Action	Rule
1	`Extract ttl_epoch_ms`	Reject if now_ms > ttl_epoch_ms
2	`Extract signature_format`	Dispatch verification accordingly. DO NOT assume base64.
3	`Extract signing_provider`	Determines which PQC algorithm to use for verification.
4	`Install pypqc`	pip install pypqc on Velos side.
5	`Verify content_hash`	Recompute from payload. Exclude: signature, public_key, signature_format, signature_algorithm, content_hash itself.
6	`Check decision field`	BLOCK = OMNIX has vetoed. Velos must not proceed.
7	`Read veto_chain`	Identify which specific gate(s) triggered the block.
8	`Reject signature_format NONE`	Treat as invalid receipt — no integrity guarantee.

15. Example — Full BLOCK Receipt (Trading Domain)


```
{
"receipt_id": "OMNIX-A3F7C1D92B0E",
"timestamp": "2026-04-09T14:23:01.847291+00:00",
"issued_at_ms": 1744208581847,
"ttl_epoch_ms": 1744208611847,
"ttl_ms": 30000,
"asset": "BTC/USDT",
"decision": "BLOCK",
"veto_chain": [
"AML: PASS -- volume_score=12.4 frequency_score=0.3",
"FRAUD: PASS -- sentiment_score=0.12 reversal_risk=0.04",
"SHARIA: BLOCK -- gharar_score=0.81 exceeds threshold 0.35",
"CAG: SKIPPED (upstream veto)"
],
"policy_version": "6.5.4e",
"engine_version": "6.5.4e",
"prev_hash": "a9f3c1d4e8b2fc71...",
"signing_provider": "dilithium3",
"sharia_compliance": {
"admitted": false,
"evaluation_state": "EVALUATED",
"violation": "GHARAR_EXCESSIVE",
"gharar_score": 0.81,
"debt_ratio": 0.0
},
"content_hash": "3f8e7a2c91bd...",
"signature": "BASE64_DILITHIUM3_SIGNATURE==",
"signature_algorithm": "Dilithium-3 (ML-DSA-65)",
"signature_format": "base64_pqc",
"public_key": "BASE64_PUBLIC_KEY=="
}
```


16. Backward Compatibility

Schema Version	Released	Changes
Schema v6.5.4e (this document)	2026-04-09	Added issued_at_ms, ttl_epoch_ms, ttl_ms, signature_format. Verifier now checks TTL expiry and exposes is_expired, age_ms.
Pre-v6.5.4e (legacy)	—	timestamp only. No TTL fields. signature_format absent. Assume base64_pqc if public_key present, else hex_sha256_fallback.

Document prepared by Harold Nunes — OMNIX QUANTUM LTD

Eureka GCC Dubai 2026 Semifinalist | Recaudando $500K pre-seed @ $3M valuation

OMNIX QUANTUM LTD — United Kingdom | omnixquantum.com